Guia Prático do Aluno — Aula 7

Empacotamento, Conteinerização e Orquestração com Spring Boot 4.1.x, PostgreSQL e Docker Compose

Este documento é o seu guia prático oficial de acompanhamento para a **Aula 7: Empacotamento, Containers e Docker Compose**. Siga os laboratórios em sequência (Lab 0 ao Lab 4) conforme apresentados no material de apoio e projecções do instrutor.

1. Configuração do Terminal e Pré-Requisitos

> 🛠 **Padronização do Terminal:** No Windows, utilize sempre o terminal Git Bash integrado no IntelliJ IDEA (Settings > Tools > Terminal > Shell path: bash.exe). No macOS ou Linux, utilize o Terminal nativo (Zsh/Bash). Isso garante paridade total nos comandos executados.

Antes de iniciar os laboratórios, confirme que as seguintes portas estão livres na sua máquina local:
- **Porta 5432:** Reservada para o container do banco de dados PostgreSQL.
- **Porta 8081:** Reservada para o container da API MyMovies (evita conflitos com a 8080 nativa).
- **Porta 8080:** Utilizada apenas durante a execução standalone local no Lab 0.

LAB 1: PostgreSQL Solitário em Container

Objetivo: Instanciar o banco de dados PostgreSQL 16 de forma isolada no Docker Compose, validando a integridade das portas, volume e healthcheck.

Passo 1.1: Criar a Especificação compose.yaml (Serviço db)

Crie o arquivo compose.yaml na raiz do projeto contendo a definição do banco de dados relacional:

```
name: mymovies-aula7

services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: mymovies
      POSTGRES_USER: ${POSTGRES_USER:-mymovies_lab}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-mymovies_pass}
    ports:
      - "127.0.0.1:5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -h 127.0.0.1 -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 10s

volumes:
  pgdata:
```

Passo 1.2: Subir e Inspecionar o Container do Banco

Execute o comando para iniciar apenas o serviço de banco de dados e acompanhar o status de saúde (healthcheck):

```
# Subir o banco de dados em segundo plano:
docker compose up -d db

# Inspecionar o status do container (aguarde o status '(healthy)'):
docker compose ps

# Verificar os logs de inicialização do PostgreSQL:
docker compose logs db
```

Status (DoD)	Critério de Sucesso do Laboratório
[ ] Concluído	Arquivo compose.yaml criado com serviço db, volume pgdata e healthcheck
[ ] Concluído	Container db em execução e reportando status (healthy) no docker compose ps
[ ] Concluído	Porta 5432 mapeada para 127.0.0.1 sem conflito com banco nativo